Clase asincrónica 22/10

— Inspección de tu cluster + auto-escalado (individual)

Contexto: segunda clase asincrónica del bloque 09. Continúa la del jueves 15/10 (clase-asincronica-intro.md — conceptos de K8s + K3s + "Hola Mundo") y cierra dos pendientes: las preguntas de cierre de la clase networking-kubernetes-y-dc.md (09.1) y el taller de auto-escalado que no llegamos a hacer en la presencial del 20/10.

Bloque: Redes, Gestión y Operación — GOYS Modalidad: individual — cada uno en su propia VM (mismas specs que pedimos para el 15/10). Duración estimada: ~2 h Entrega: un archivo .md con capturas de pantalla + respuestas. Se responde con evidencia: sin captura, no hay prueba.

Cómo recorrer esta clase

Paso	Actividad	Tiempo
1	Ver el video ⭐ *K3s el Kubernetes más simple*	15 min
2	(Re)leer las secciones ancla del material de la 09.1	20 min
3	Bloque 0 — Punto de partida: VM + K3s Ready	15 min
4	Bloque A — Inspección de la red (las 4 preguntas de cierre)	35 min
5	Bloque B — Auto-escalado con HPA (el taller que no hicimos)	50 min
6	Entregable + respuestas	15 min

💡 Material principal = la lectura y la práctica. El video es el enganche. Si algo no cierra con el video, la lectura y las capturas lo despejan. No hay atajos: sentá las bases.

Reglas de la actividad (leelas antes de empezar)

1. Es individual. No hay grupo: cada uno arma, ejecuta y responde en su VM.

Toda captura lleva tu identidad. Al principio de cada bloque ejecutá:
`echo "=== APELLIDO, Nombre — legajo XXXXX ==="`

2.
3. y dejá esa línea visible en la captura. Es tu firma: sin ella, la captura no cuenta.
4. Los YAML que entregues llevan un comentario con tu nombre/legajo arriba de todo.
5. Formato de entrega: un archivo `clase-09-2-apellido-nombre.md` con las capturas embebidas y las respuestas. Todo en un solo archivo, subido al Drive de entregas + confirmación por mail (los dos pasos, abajo en el checklist). Entrega: martes 27/10 23:59.
6. Sin recetas copiadas entre compañeros. Las salidas de TU cluster son únicas (IPs, podCIDR, tiempos). Si dos entregas tienen la misma salida exacta, ambas vuelven.

Qué necesitás

Requisito	Detalle
VM	Igual que la pedida el 15/10: 2 vCPU · 3–4 GB RAM · 20 GB disco · red en PUENTE (bridge) — Linux solo CLI (Rocky / openSUSE / Ubuntu Server).
K3s	Instalado de la asincrónica del 15/10. ¿No lo tenés? Mirá el *Camino rápido* del Bloque 0: 10 min y estás al día.
Internet	La VM necesita salir a internet (pull de imágenes en Docker Hub y `registry.k8s.io`).

Pacienca	El Bloque B tiene esperas de ~5 min (ventana de estabilización del HPA). Es parte del aprendizaje, no es muerto.

Paso 1 — Video (15 min)

Uso	Video	Canal	Dur.
⭐ El video de esta clase	K3s el Kubernetes más simple	Nivel Epsilon (ene 2026, 🇪🇸)	~16 min

Dos preguntas para tener en la cabeza mientras lo ves (van en el entregable):

1. El video muestra lo simple que es instalar K3s. ¿Qué se esconde detrás de esa simplicidad — qué componentes del Kubernetes "completo" K3s empaqueta en un solo binario?
2. ¿Por qué esa simplicidad lo hace ideal para edge / laboratorios y no te haría descartarlo por eso de cara a aprender los internals de K8s?

Secciones ancla (lo que ya tenés, releelo si no te acordás)

Para el Bloque A (red)	Para el Bloque B (HPA)
clases/clase-09-1-networking-kubernetes/networking-kubernetes-y-dc.md → §1.1–1.5 (4 reglas, 3 planos, Service/CNI), §2.1–2.5 (underlay/overlay, BGP, MTU)	clases/clase-09-k3s/taller-auto-scaling.md → Bloques 1–3 (requests, HPA, scale-down)
clases/clase-09-k3s/clase-asincronica-intro.md → §1.1–1.2 (reconciliation loop, componentes)	clases/clase-09-k3s/clase-asincronica-intro.md → §3.4 (self-healing)

Bloque 0 — Punto de partida (15 min)

Sin esto, nada de lo demás funciona. Verificá que la VM y el cluster están sanos.

```
echo "=== APELLIDO, Nombre — legajo XXXXX ==="   # tu firma para la captura 1

hostname -I                                     # anotá la IP de la VM
kubectl get nodes                               # DEBE decir Ready
kubectl get pods -A                             # coredns, traefik, metrics-server...
kubectl top nodes                               # DEBE devolver números, no error
```

✅ Captura 1: kubectl get nodes (nodo Ready) + kubectl top nodes, con tu firma visible.

⚠️ Si kubectl no existe o pedís sudo: usá sudo k3s kubectl ..., o re-hacé el paso de configuración de la asincrónica (§3.2): copiá /etc/rancher/k3s/k3s.yaml a ~/.kube/config.

¿No tenés la VM o K3s está caído? — Camino rápido (10 min)

1. Creá la VM con las specs de la tabla de arriba (Linux solo CLI, red bridge).

Instalá K3s — un solo comando:

```
curl -sfL https://get.k3s.io | sh -
```

Configurá kubectl sin sudo:

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $(id -u):$(id -g) ~/.kube/config
chmod 600 ~/.kube/config
kubectl get nodes       # esperá ~60 s a que pase a Ready
```

2. Volvé a la Verificación del Bloque 0.

Detalles (Rocky 10, firewalld, SELinux, imágenes pre-armadas): clase-asincronica-intro.md §3. Desinstalación y arranque limpio: sudo /usr/local/bin/k3s-uninstall.sh.

Síntoma	Causa	Solución
Nodo NotReady	CNI/flannel no arrancó	sudo journalctl -u k3s -n 100 --no-pager; deshabilitá firewalld
Pods en ContainerCreating eterno	SELinux (Rocky/RHEL)	getenforce; sudo setenforce 0 (lab)

kubectl top da error	metrics-server tarda / caído	kubectl get pods -n kube-system y esperá 1 min

---

Bloque A — Inspección de la red: las 4 preguntas de cierre (35 min)

Esto es lo que quedó pendiente de la clase 09.1 (networking-kubernetes-y-dc.md). No es "hacer cosas": es ver con los ojos propios la teoría. Cada sub-bloque = comandos + captura + pregunta. Las 4 preguntas se responden por escrito en el entregable.

A1 — Los tres planos de direcciones (10 min)

```
echo "=== APELLIDO, Nombre — legajo XXXXX ===" # tu firma

kubectl get nodes -o wide # PLANO NODE: la IP de la máquina
kubectl get nodes -o jsonpath='{range .items[*]}{.metadata.name}{"
podCIDR="}{.spec.podCIDR}{"\n"}{end}'
kubectl get pods -A -o wide | head # PLANO POD: 10.42.x (efímeras)
kubectl get svc -A # PLANO SERVICE: 10.43.x (VIPs virtuales)
```

✅ Captura 2: las cuatro salidas juntas, con tu firma.

📖 Por qué existen tres planos y por qué no deben solaparse con la red de la organización: networking-kubernetes-y-dc.md §1.3.

A2 — La VIP que "no existe" (10 min)

```
echo "=== APELLIDO, Nombre — legajo XXXXX ==="

kubectl get svc -n kube-system kube-dns # ClusterIP 10.43.0.10
ping -c 2 10.43.0.10 # NO responde — probalo y dejá la evidencia
kubectl get endpoints -n kube-system kube-dns # las IPs REALES detrás de la VIP

kubectl run dnstest --image=busybox:1.28 --restart=Never --rm -it -- \
  nslookup kube-dns.kube-system.svc.cluster.local # el DNS interno resuelve nombres
```

✅ Captura 3: el ping sin respuesta + kubectl get endpoints (que sí muestra IPs), con tu firma.

📖 Traducción VIP → IP de pod: kube-proxy (DNAT). networking-kubernetes-y-dc.md §1.4.

A3 — ¿Quién conoce la ruta entre el nodo y los pods? (8 min)

```
echo "=== APELLIDO, Nombre — legajo XXXXX ==="

ip -brief address                          # la IP de TU nodo (el plano node, ej. 192.168.0.19)
kubectl get pods -A -o wide | head         # los pods en 10.42.x
ip route | head                            # la ruta por defecto del nodo

sudo iptables -t nat -L KUBE-SERVICES -n | head    # el NAT de los Services (kube-proxy)
```

✅ Captura 4: IP del nodo + IPs de pods + al menos una regla KUBE-SERVICES, con tu firma.

📖 Si el cluster fuera multi-nodo y otro host del DC quisiera llegar a un pod: sin BGP del CNI (Calico/Cilium) no puede. networking-kubernetes-y-dc.md §2.3.

A4 — El CNI en uso y la MTU (7 min)

```
echo "=== APELLIDO, Nombre — legajo XXXXX ==="

ls /etc/cni/net.d/ 2>/dev/null                  # path estándar de CNI (en K3s suele venir VACÍO)
sudo ls /var/lib/rancher/k3s/agent/etc/cni/net.d/   # el que usa K3s de verdad (flannel por defecto)
sudo cat /var/lib/rancher/k3s/agent/etc/cni/net.d/10-flannel.conflist 2>/dev/null | head -20
ip -o link show | grep -E 'cni0|flannel'        # las interfaces del overlay
ip link show flannel.1 2>/dev/null | head -1    # MTU del TÚNEL VXLAN (la que limita entre nodos)
ip link show cni0 2>/dev/null | head -1         # MTU del bridge local (la que ven tus pods)
```

✅ Captura 5: el CNI configurado + la MTU de flannel.1 y de cni0, con tu firma.

🔎 Ojo, ahí hay una trampa: flannel.1 (el túnel VXLAN) va a mostrar ~1450 y cni0 (el bridge local) 1500. En single-node tus pods hablan por cni0 sin encapsular, pero entre nodos el límite lo pone flannel.1. Fijate bien qué interfaz reporta cada salida antes de responder — esa es justamente la pregunta 4.

✍️ Las 4 preguntas de cierre del Bloque A (respuestas en el entregable)

1. ¿Cuál es la diferencia entre la IP de un pod (10.42.x) y la de un Service (10.43.x)? ¿Por qué una responde al ping y la otra no?
2. El Service kube-dns tiene una sola VIP, pero kubectl get endpoints muestra varias IPs. ¿Quién convierte una en la otra?

3. Tu nodo tiene IP 192.168.x.x y los pods 10.42.x: ¿quién conoce la ruta entre ambos? ¿Qué pasaría si otro host del DC (fuera del cluster) quisiera llegar a un pod?
4. ¿Qué MTU efectiva tienen los paquetes de pod→pod con flannel (tu captura 5)? ¿Qué overhead agrega la encapsulación VXLAN y qué síntoma típico produce un MTU mal ajustado?

Bloque B — Auto-escalado con HPA (50 min) — el taller que no hicimos

La pregunta: ya saben escalar a mano (kubectl scale). ¿Puede Kubernetes decidir solo cuántas réplicas poner, según la carga? Sí: HPA (HorizontalPodAutoscaler). Este es el taller-auto-scaling.md, corriendo individualmente en tu VM.

B0 — Verificación previa (sin esto el HPA no anda) (5 min)

```
echo "=== APELLIDO, Nombre — legajo XXXXX ==="

kubectl get nodes
kubectl get pods -n kube-system | grep metrics-server   # DEBE estar Running
kubectl top nodes                                       # DEBE responder con números
```

⚠️ Si kubectl top falla, pará acá. Sin métricas, el HPA queda en <unknown> y no escala.

✅ Captura 6: metrics-server Running + kubectl top nodes, con tu firma.

B1 — Desplegar la app con CPU requests (15 min)

Creá hpa-demo.yaml (con tu nombre/legajo en el comentario):

```
nano hpa-demo.yaml
```

```
# APPELLIDO, Nombre — legajo XXXXX
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels: { app: web }
  template:
    metadata:
```

```
    labels: { app: web }
  spec:
    containers:
      - name: web
        image: registry.k8s.io/hpa-example
        ports:
          - containerPort: 80
        resources:
          # 👇 SIN esto el HPA NO puede calcular el % de uso. Es obligatorio.
          requests:
            cpu: 200m
---
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector: { app: web }
  ports:
    - port: 80
      targetPort: 80
```

```
kubectl apply -f hpa-demo.yaml
kubectl get pods -o wide
kubectl get endpoints web          # DEBE mostrar la IP del pod

# Repaso: escalar A MANO (después volvemos a 1)
kubectl scale deployment web --replicas=3
kubectl get pods -l app=web
kubectl scale deployment web --replicas=1
```

💡 ¿Por qué requests? El HPA no mide CPU absoluta: mide % respecto del request. Pod pide 200m y consume 100m → 50%. Sin request no hay denominador → no escala.

B2 — Crear el HPA y verlo escalar (20 min)

Creá hpa.yaml:

```
# APPELLIDO, Nombre — legajo XXXXX
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web
```

```
  minReplicas: 1                # piso
  maxReplicas: 5                # techo
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50  # objetivo: CPU promedio en 50% del request
```

```
kubectl apply -f hpa.yaml
kubectl get hpa
```

✅ Captura 7 — HPA en reposo: `kubectl get hpa` con TARGETS en 0%/50% (tarda ~30–60 s en mostrar el %; si dice `<unknown>`, esperá).

Lanzá la carga dentro del cluster (en una terminal/secundaria o con `&`):

```
kubectl run load-generator --image=busybox:1.28 --restart=Never -- \
  /bin/sh -c "while true; do wget -q -O- http://web; done"
```

Mirá el HPA trabajar:

```
for i in 1 2 3 4 5 6; do sleep 30; kubectl get hpa; echo "pods: $(kubectl get pods -l app=web --no-headers | wc -l)"; done
```

✅ Captura 8 — HPA bajo carga: `kubectl get hpa` con el % alto y REPLICAS > 1 (en cluster de referencia llegó a 381%/50% y escaló 1 → 5).

```
kubectl describe hpa web      # sección "Events": "New size: 5; reason: cpu utilization above target"
kubectl top pods -l app=web
```

✅ Captura 9: `kubectl describe hpa web` con el evento de escalado a la vista.

🎉 Eso es auto-escalado. Nadie tocó `replicas`: K8s leyó la métrica, la comparó con el objetivo (50%) y agregó pods solo hasta el techo. El mismo *reconciliation loop* de la asincrónica del 15/10, ahora alimentado por métricas.

B3 — Bajar la carga y ver el scale-down (10 min)

```
kubectl delete pod load-generator
kubectl get hpa -w      # (Ctrl+C para salir) → esperá ~5 min: REPLICAS baja de 5 → 1
```

✅ Captura 10: kubectl get hpa después de la ventana de estabilización, con REPLICAS bajando o ya en 1. Paciencia: la ventana por defecto es de 5 minutos — no es un error.

💡 Sube rápido, baja lento: escalar hacia arriba es urgencia; hacia abajo es prudencia. Es el mismo criterio que un termostato que no apaga la caldera al primer grado de más.

✍️ Preguntas de cierre del Bloque B (respuestas en el entregable)

5. ¿Por qué el HPA necesita que el Deployment tenga resources.requests.cpu?
6. Si ponés maxReplicas: 5 y la carga sigue subiendo, ¿qué pasa? ¿Por qué no escala más?
7. ¿Por qué el scale-down tarda minutos y el scale-up no?

Cierre integrador — 2 preguntas finales

8. El HPA crea pods nuevos cuando sube la carga. Volvé a la captura 2 (A2): esos pods nuevos reciben IP del rango 10.42.x y hay que poder llegar a ellos desde el Service. ¿Qué componente del cluster se actualiza solo cuando nacen o mueren pods? Y relacionado: ¿qué problema de planificación de direcciones del DC se agranda cuando el número de pods cambia todo el tiempo? (networking-kubernetes-y-dc.md §2.2)
9. ¿Cuál es la diferencia entre lo que decidís vos con kubectl scale y lo que decide el HPA? En los dos casos, ¿quién "sostiene" esa intención?

Entregable — checklist final

Archivo: clase-09-2-apellido-nombre.md (capturas embebidas + respuestas numeradas 1–9).

#	Evidencia	Qué tiene que verse
1	Bloque 0	kubectl get nodes Ready + kubectl top nodes

2	A1	3 planos: nodes -o wide, podCIDR, pods -o wide, svc -A
3	A2	ping al ClusterIP sin respuesta + get endpoints con IPs reales
4	A3	IP del nodo + IPs de pods + regla KUBE-SERVICES
5	A4	CNI configurado + MTU del overlay
6	B0	metrics-server Running + kubectl top nodes
7	B2	kubectl get hpa en reposo (0%/50%)
8	B2	kubectl get hpa bajo carga (REPLICAS > 1)
9	B2	kubectl describe hpa web con el evento de escalado
10	B3	kubectl get hpa tras la ventana: scale-down (5 → 1)
📄	YAMLs	hpa-demo.yaml + hpa.yaml (comentario con nombre/legajo)
✍️	Respuestas	1–4 (red) · 5–7 (HPA) · 8–9 (integradoras) · + 2 del video

Toda captura lleva tu firma (echo "=== APELLIDO, Nombre — legajo XXXXX ===" visible). Sin firma = captura de cualquiera = no cuenta.

📥 Cómo se entrega (2 pasos, ambos obligatorios · hasta el martes 27/10, 23:59)

Paso 1 — Drive: subí tu clase-09-2-apellido-nombre.md acá:

📂 https://drive.google.com/drive/folders/1s6J5ZK7T1TDC5hv7RFisRQpaXRilUgSX?usp=drive_link

Paso 2 — Confirmación por mail (para que no haya dudas de quién entregó qué):

- Asunto: [GOYS] Confirmo entrega asincrónica 22/10 — Apellido, Nombre (legajo XXXXX)

Cuerpo:
Nombre y Apellido:
Legajo:
Grupo:
Archivo subido: clase-09-2-apellido-nombre.md

-

⚠️ Sin el mail de confirmación, la entrega no cuenta. El Drive solo dice "hay un archivo"; tu mail de confirmación es lo que lo ata a tu legajo.

Troubleshooting general

Síntoma	Causa probable	Solución
Nodo NotReady	CNI/flannel no arrancó	sudo journalctl -u k3s -n 100 --no-pager
ping 10.43.x no responde	Es correcto: la VIP no está en ninguna interfaz	No lo arregles — es la respuesta a la pregunta 1
kubectl top da error	metrics-server caído/tardando	kubectl -n kube-system logs deploy/metrics-server
HPA TARGETS: <unknown>	sin métricas todavía	esperá 1 min; verificá metrics-server
HPA no escala con carga	falta requests.cpu	agregalo y re-aplicá hpa-demo.yaml
HPA no escala	load-generator sin ruta al Service	kubectl get endpoints web tiene que tener IP
Pods Pending al escalar	recursos del nodo (single-node)	bajá maxReplicas o el requests
curl :30080 no responde	red NAT / firewall	usá bridge; probá primero dentro de la VM

Referencias

De la clase 09.1 (las preguntas de cierre vienen de ahí):

- Dutt — *Cloud Native Data Center Networking* (Cap. 2–3 spine-leaf, 7–8 VXLAN/EVPN, 14 BGP) — la base de la Etapa 1, releída con ojos de K8s.
- Kubernetes.io → *Cluster Networking* y *Services, Load Balancing, and Networking*.
- CNI spec — https://github.com/containernetworking/cni
- Cilium — *Kubernetes Networking Deep Dive* / docs.cilium.io
- clases/clase-10-muestra-tp2/README.md — Network Policies + Cilium (el "con las manos").

De esta asincrónica:

- clases/clase-09-k3s/clase-asincronica-intro.md — la asincrónica del 15/10 (base del cluster).
- clases/clase-09-1-networking-kubernetes/networking-kubernetes-y-dc.md — teoría del Bloque A (§1 y §2).
- clases/clase-09-k3s/taller-auto-scaling.md — el taller original del Bloque B.
- K3s el Kubernetes más simple — Nivel Epsilon (video de la clase).
- Kubernetes.io → *Tasks* → *Horizontal Pod Autoscaling*.
- docs.k3s.io → *Installation / Architecture*.